EST. 2026 · BUENOS AIRES · V1.0

— UN COMPILADOR EN CÓDIGO MORSE —

Morse·Lab

/ LÉXICO · SINTÁCTICO · SEMÁNTICO · INTÉRPRETE · TTS /

Informe del TP integrador

Lenguajes Formales y Compiladores

Python 3.12 · 74 tests · todo a mano

GITHUB.COM/MARTINPULI/COMPILADOR · COMPILADOR-SWART.VERCEL.APP

Lenguaje: MorseLang — lenguaje imperativo cuyo código fuente se escribe en código Morse.
Implementación: Python 3.12, todo a mano (sin parser generators), 74 tests automatizados.

> *Para una vista interactiva de cada Parte, abrir el Studio y usar la pestaña **TP Inspector**:* `streamlit run studio/app.py`.

01 Definición del lenguaje

Descripción informal

MorseLang es un lenguaje imperativo de propósito general (nivel intermedio). Soporta variables tipadas (NUM, TEXTO, BOOL), aritmética entera, comparaciones, `SI/SINO`, `MIENTRAS` y `MOSTRAR`. Su rasgo distintivo es que el código fuente se escribe en código Morse: cada keyword, identificador, número y string es una secuencia de `.` y `-`.

Alfabeto

En lenguajes formales, el **alfabeto** Σ es el conjunto de *símbolos atómicos* que pueden aparecer en un programa fuente — los caracteres que el lexer ve directamente en el archivo. Para MorseLang:

```
Σ = { '.', '-', ' ', '/', '\n', '"' }
```

Sólo seis caracteres. Las letras `A`, `B`, ..., `Z` y los dígitos `0` – `9` **no están en el alfabeto** porque no aparecen literalmente en un archivo `.morse`: emergen recién cuando el lexer decodifica secuencias Morse (`.-` se vuelve `A`, `-----` se vuelve `0`, etc.). Por eso el alfabeto es tan chico — todo lo demás se construye combinando puntos y rayas.

Cada símbolo del alfabeto cumple un rol específico:

SÍMBOLO	ROL
`.` `-`	Componentes atómicos de cada letra/dígito Morse.
` `(un espacio)	Separa letras dentro de una misma palabra Morse.
` `(tres espacios) o `/`	Separa palabras / tokens.
`\n` (salto de línea)	Termina una sentencia.
`"`	Delimita literales de texto (su contenido también es Morse).

Tokens

CATEGORÍA	EJEMPLOS
Keyword	VAR, MOSTRAR, SI, SINO, MIENTRAS, FIN, VERDADERO, FALSO
Operador (palabra Morse)	MAS, MENOS, POR, DIV, IGUAL, DIST, MEN, MAY, ASIG
Identificador	A–Z, ≥1 letra
Número	secuencia de dígitos Morse
Texto	`"..."` con contenido Morse
Estructural	NEWLINE, EOF

Gramática (EBNF)

```
programa     ::= { sentencia } ;
sentencia    ::= declaracion | asignacion | mostrar | si | mientras ;
declaracion  ::= "VAR" identificador "ASIG" expresion "\n" ;
asignacion   ::= identificador "ASIG" expresion "\n" ;
mostrar      ::= "MOSTRAR" expresion "\n" ;
si           ::= "SI" expresion "\n" { sentencia } [ "SINO" "\n" { sentencia } ] "FIN" "\n" ;
mientras     ::= "MIENTRAS" expresion "\n" { sentencia } "FIN" "\n" ;

expresion    ::= comparacion ;
comparacion  ::= aritmetica [ ("IGUAL"|"DIST"|"MEN"|"MAY") aritmetica ] ;
aritmetica   ::= termino { ("MAS"|"MENOS") termino } ;
termino      ::= factor { ("POR"|"DIV") factor } ;
factor       ::= numero | texto | "VERDADERO" | "FALSO" | identificador
              | "(" expresion ")" ;
```

Ejemplo de derivación

Para `VAR X ASIG 10`:

```
sentencia ⇒ declaracion
         ⇒ "VAR" identificador "ASIG" expresion
         ⇒ "VAR" "X" "ASIG" numero
         ⇒ "VAR" "X" "ASIG" "10"
```

02 Análisis léxico

Estrategia

Dos pasos: (1) decodificación Morse → texto, (2) clasificación del texto en tokens. La separación mantiene el AFD de la primera fase pequeño y testeable.

Cómo se reconoce un programa Morse

El lexer no intenta reconocer las palabras del lenguaje directamente sobre los puntos y rayas — eso explotaría la cantidad de estados del autómata. En cambio funciona en dos pasos sobre el mismo stream de entrada:

Paso 1 — Decodificar Morse → texto plano

Recorre el archivo y agrupa caracteres usando los separadores definidos en el alfabeto:

- Un espacio dentro de una palabra Morse: separa **letras**.
- Tres espacios o `/`: separan **tokens** (palabras enteras del lenguaje).
- Salto de línea: separa **sentencias**.
- Comillas: delimitan un **literal de texto** (su contenido se decodifica igual).

Cada secuencia de letras Morse (separadas por un solo espacio) se busca en una tabla `{ '.-': 'A', '-...': 'B', ..., '-----': '0', ..., '----.': '9' }` y se concatena en una palabra ASCII.

Para la entrada `...- .- .-. / -..- / .- ... .. --. / .---- -----` el paso 1 produce:

```
VAR  X  ASIG  10
```

Paso 2 — Clasificar palabras en tokens tipados

Sobre el texto decodificado, cada palabra ASCII se reconoce contra una tabla de keywords / operadores y una expresión regular para identificadores y números:

PALABRA DECODIFICADA	TOKEN EMITIDO
`VAR`, `MOSTRAR`, `SI`, `SINO`, `MIENTRAS`, `FIN`, `VERDADERO`, `FALSO`	KEYWORD del tipo correspondiente
`MAS`, `MENOS`, `POR`, `DIV`, `IGUAL`, `DIST`, `MEN`, `MAY`, `ASIG`	OPERATOR del tipo correspondiente
Cualquier secuencia `[A-Z]+` no reservada	IDENT
Cualquier secuencia `[0-9]+`	NUMBER
Contenido entre `"..."` (decodificado)	STRING

Los strings preservan espacios internos cuando dentro de las comillas hay tres espacios o `/` entre palabras Morse — eso permite escribir literales como `"HOLA MUNDO"` con un espacio real entre las dos palabras.

Salida final: una secuencia plana de tokens tipados con su línea de origen, lista para que el parser construya el AST.

Expresiones regulares (sobre el texto decodificado)

TOKEN	REGEX
KEYWORD/OPERATOR	`VAR|MOSTRAR|SI|...|ASIG`
IDENT	`[A-Z]+`
NUMBER	`[0-9]+`
STRING	`"[^"]*"`
NEWLINE	`\n`

AFD del decodificador Morse

```
        ┌─────────┐
start ─▶│   S0    │── '.' or '-' ──▶┐
        └─────────┘                 ▼
                                ┌─────────┐
                                │   S1    │── '.' or '-' ──▶ S1
                                │ accum   │
                                └─────────┘
                                │  ' '    │  '/' / '\n' / '"'
                                ▼         ▼
                            emit letter   emit separator / start string
```

Implementación: `morselang/lexer.py`. Visualización en vivo: TP Inspector → pestaña 2.

03 Análisis sintáctico

Tipo

Recursivo descendente, **LL(1)**.

Justificación

Cada no-terminal se decide con un único token de lookahead. Las sentencias se discriminan por keyword inicial (VAR/MOSTRAR/SI/MIENTRAS) o IDENT (asignación). Las expresiones usan precedencia por niveles (`comparacion` → `aritmetica` → `termino` → `factor`) para evitar recursión por izquierda. Es el método más didáctico para un TP y se implementa en pocas líneas por no-terminal — alineado con el requisito de hacer todo a mano.

Implementación

`morselang/parser.py`. Construye un AST de dataclasses definidas en `morselang/ast_nodes.py`.

Ejemplo

Para `VAR X ASIG 10 MAS 5`:

```
Programa
└── Declaracion(name='X')
    └── BinOp('MAS')
        ├── NumeroLit(10)
        └── NumeroLit(5)
```

Visualización en vivo: TP Inspector → pestaña 3.

---

04 Tabla de símbolos

Estructura

`dict` con clave = nombre de la variable y valor = `SymbolInfo(tipo, valor, linea_declaracion)`. Métodos: `declarar`, `asignar`, `consultar`, `existe`, `snapshot`. Un único ámbito global.

Estrategia

- `declarar(name, tipo, line)` — falla si ya existe.
- `asignar(name, valor)` — falla si no fue declarada.
- `consultar(name)` — falla si no existe.

Ejemplo de uso (factorial.morse)

LÍNEA	TABLA
1 (VAR N=5)	`{N: NUM=5}`
2 (VAR R=1)	`{N: NUM=5, R: NUM=1}`
iter 1	`{N: NUM=4, R: NUM=5}`
iter 2	`{N: NUM=3, R: NUM=20}`
iter 3	`{N: NUM=2, R: NUM=60}`
iter 4	`{N: NUM=1, R: NUM=120}`
iter 5	`{N: NUM=0, R: NUM=120}`
7 (MOSTRAR R)	`{N: NUM=0, R: NUM=120}` → imprime 120

Visualización en vivo: TP Inspector → pestaña 4.

05 Análisis semántico

Reglas

1. Variable declarada antes de usarse.
2. No redeclaración.
3. Aritmética requiere ambos operandos NUM.
4. Comparación requiere operandos del mismo tipo.
5. Condición de SI / MIENTRAS debe ser BOOL.
6. División por literal `0` rechazada en compilación.
7. División por variable `0` se detecta en runtime (`RuntimeError_`).

Errores

PROGRAMA	MENSAJE
`error_no_declarada.morse`	`Línea 1: variable 'Z' no declarada`
`error_redeclaracion.morse`	`Línea 2: redeclaración de 'X' (declarada originalmente en línea 1)`
`error_division_cero.morse`	`Línea 1: división por cero`

Visualización en vivo: TP Inspector → pestaña 5.

06 Síntesis — intérprete + ElevenLabs

Se eligió **intérprete tree-walking** como fase de síntesis (Parte 6 del TP — opción "Intérprete del lenguaje"). El intérprete recorre el AST, mantiene una tabla de símbolos en runtime y captura cada `MOSTRAR` en `interp.output`.

Como cierre vistoso, la salida acumulada se envía a la **API de ElevenLabs** para producir un `output.mp3` que narra el resultado con voz humana en español.

```
archivo.morse → Lexer → Parser → AST → Semántico → Intérprete → output.mp3
```

CLI:

```
python main.py examples/factorial.morse        # solo ejecuta
python main.py examples/factorial.morse --tts  # + narración ElevenLabs
```

Studio:

```
streamlit run studio/app.py
```

07 Uso de IA

La consigna del TP permite explícitamente el uso de IA siempre que esté documentado. Esta sección resume lo más relevante; el detalle completo (prompts textuales, lecciones, decisiones descartadas) vive en `docs/uso_ia.md`.

Herramienta

Claude Code (Anthropic) — modelo Opus 4.7 con contexto de 1M tokens, en VS Code. Sub-agentes Haiku 4.5 despachados desde el agente principal para tareas mecánicas (correr `pytest`, hacer commits) con prompts cerrados, evitando contaminar el contexto principal. **MCP Playwright** para verificación visual end-to-end del frontend.

No se usaron otras herramientas (ChatGPT, Copilot, etc.).

Workflow

El desarrollo siguió un patrón disciplinado:

1. **Brainstorming** del lenguaje y la arquitectura (preguntas guiadas, no implementación).
2. **Spec** en markdown con todas las decisiones (alfabeto, gramática, alcance, riesgos).
3. **Plan de implementación** TDD detallado con todo el código por adelantado, en 13 tareas bite-sized.
4. **Ejecución por subagentes** — uno por tarea, con review automático entre tareas.
5. **Code review final** sobre el branch completo.
6. **Verificación visual** con Playwright (boot del server, navegación, screenshots por vista).

Prompts clave (selección)

> *"Una idea innovadora quiero" → arrancó la exploración, derivó en tres opciones (PromptLang / DanzaLang / FinLang) que después se descartaron.*
>
> *"En morse podra ser? y que este conectado a la api de elevenlabs y con eso traduzca a algo pa poder programar" → idea final: lenguaje fuente en código Morse + ElevenLabs como cierre vistoso de la fase de síntesis.*
>
> *"ME GUSTARI9A QUE LE HAGAS UNA VISUAL PARA PROBARLO, Y QUE PEUDAS GRABAR UN AUDIO QEU ENTIENDA EL CODIGO MORSE Y LOI PONGA EN CODIGO" → MORSE.LAB con cuatro vistas + decoder de audio DSP-puro.*

Componentes generados con asistencia

Todo el código del compilador (`morselang/`), la UI web (`web/`), los tests (78 en total), los ejemplos en Morse, el script que renderiza este PDF y la documentación. Todo pasó por revisión humana antes del commit.

Errores detectados en respuestas de IA y correcciones

#	ERROR	CORRECCIÓN
1	Mensaje de error semántico para tipo aritmético no contenía la palabra "tipo", rompiendo un test.	Se prependió `"tipo incompatible — "` en `morselang/semantic.py:_binop`.
2	El lexer colapsaba espacios dentro de strings (`"HOLA MUNDO"` → `"HOLAMUNDO"`), violando la regla `texto` de la gramática.	Se reescribió `_emit_string` para separar el contenido por triple-espacio o `/` y unirlo con un espacio real.
3	El primer test del decoder de audio asertaba un rango muy estrecho de WPM. La ventana RMS de 10 ms inflaba la duración detectada.	Se redujo la ventana a 3 ms y se relajó la tolerancia del test.
4	El layout inicial de MORSE.LAB tenía el contenido principal aplastado a 56 px porque el CSS grid asignaba columnas en orden de aparición.	Se reescribió `body` con `grid-template-areas` para asignar columnas por nombre. Detectado con Playwright.
5	Una rama del parser para bloques anidados consumía un `FIN` extra.	Se ajustó `_parse_block` con un set explícito de `stop_tokens` y se trazaron los tokens manualmente.

Lecciones aprendidas

- **Plans con código completo > prompts iterativos.** Armar el plan con todo el código y los tests por adelantado evita el ping-pong típico con la IA.
- **Sub-agentes para tareas mecánicas.** Haiku con instrucciones cerradas es perfecto para correr pytest o commitear, sin contaminar el contexto principal.
- **TDD disciplinado.** Cada tarea empieza con un test que falla. Detectó varios casos en que la IA hubiera entregado código que no satisfacía el requisito exacto.
- **Verificación visual con Playwright.** El bug del layout no era detectable con tests unitarios — sólo el screenshot lo evidenció.
- **El reviewer agresivo paga.** El code reviewer encontró 6 issues en una pasada; 2 reales se arreglaron antes del merge.